Smishing Triage — Technical Writeup

BC3415 AI in Accounting & Finance · Individual Assignment (Option A)
Jonathan Low Seng Kian · U2321652C · 20 October 2026

Summary. A three-way classifier separates legitimate messages, marketing spam and smishing (phishing by SMS), served as a forward-it-to-check bot for a Singapore bank's customers.

- **Test performance:** catches 73.3% of smishing (95% CI 64.8–81.9%) at 80.4% precision. No scam in the test set was marked legitimate: the other 27 were called spam, so the user is still warned.
- **False alarms:** flags 0.24% of ordinary Singaporean personal messages (21 of 8,616). That figure comes from personal chat collected before 2015, close to the training ham, and was used as a selection guardrail. Genuine bank and SingPost notices fare worse: 4 of 5 were flagged.
- **Main gap:** on 60 real scams sent to people in Singapore it catches 48% and warns on 65%. Scams with a link are mostly caught; chat-style scams without one mostly are not, and receive a "no scam signals found" reply.
- **Recommendation:** not ready for customers. Fund a three-month pilot, mainly labelled local data (estimated at about S$5,000 of analyst time), with explicit go/no-go criteria (section 1).

1. Problem and business case

Smishing Triage is a forward-it-to-check scam detector for a Singapore bank's customers. A customer forwards a suspicious text, or a screenshot of one, and within seconds gets a verdict (scam, marketing spam, or no scam signals) along with the words that triggered it. This section makes the case for funding a pilot.

The problem, in money. Scams cost people in Singapore S$913.1 million across 37,308 cases in 2025, about S$24,500 per case [1], and S$410.6 million in the first half of 2026 alone [2]. Text messages are a primary way scams arrive, and filtering at the network level still lets messages through.

Where it fits. The government's ScamShield app filters suspicious SMS, and its WhatsApp bot checks forwarded messages against police reports and a classifier [4]. Smishing Triage is a complement, not a replacement. It is the bank's own layer: it runs in the bank's channel, separates marketing spam from scams, and shows the customer *which words* raised the flag. Every check also stays with the bank's fraud team, which sees new scam wording first.

Why it matters to the bank. Since 16 December 2024, a bank that breaches its anti-phishing duties under the MAS–IMDA Shared Responsibility Framework bears the full loss from a covered phishing scam, with the telco next in line [3]. Fewer customers phished means fewer losses to dispute, and less exposure wherever the bank's controls fall short. A customer who sees *why* a message was flagged also learns to spot the next one unaided.

Why three classes, not two. A standard spam filter puts a marketing blast and a credential-harvesting message in the same class. For the customer, one wastes three seconds and the other can empty an account. Separating them turns an inbox tidier into a fraud control, so success is measured by how many scams are caught (smishing recall), not by accuracy. On messages that are 83% legitimate, a model that calls everything legitimate scores 83% accuracy and catches nothing.

What the prototype delivers today, and what it does not. On held-out test data it catches 73% of scam messages, and none of the 101 test scams is marked legitimate. It runs on a laptop; a small server costs a few hundred dollars a year. It is not ready for customers. On 60 real local scams it catches 48% and warns on 65%, and 4 of 5 genuine bank and SingPost notices were flagged. Both gaps have one cause in the public training

data: its scams are British and Indian, mostly premium-rate prize texts, and its legitimate messages are personal chat (partly Singaporean) with almost no transactional SMS.

The pilot. Three months, with the money going mostly to local data. Costs are estimates.

Phase	Work	Estimated cost
Months 1–2: local data	Label about 3,000 local messages from the bank's fraud reports, customer forwards and its own genuine transactional SMS (OTPs, alerts), each checked twice: about 100 analyst-hours	about S$5,000 (at S$50 per hour)
Month 3: retrain and trial	Retrain with this pipeline (minutes), test on a fresh held-out local set, then a staff-only trial in the bank's channel	existing staff; server a few hundred S$ a year
Throughout	Scam-type labels from a paid-tier or self-hosted LLM: customer messages cannot go to a free tier that may train on them	usage-based; small at pilot volumes

Go/no-go criteria, measured on a fresh local test set never used in training:

- catch at least 80% of local scams as smishing, and warn on at least 90%;
- flag at most 1% of the bank's own genuine transactional messages;
- reply within 10 seconds, and store no message bodies.

For scale, the pilot's estimated cost is about a fifth of the average loss from one scam. The yearly benefit is roughly checks per year × share that are scams × share caught × average loss avoided; the pilot exists to measure the first three, so the case can be re-made with real numbers before any wider rollout.

How it maps to the brief. The brief's example combines review text, customer images and a joint flag. This project follows the same pattern: text classification produces the verdict, the image channel is a screenshot read by OCR, and a second LLM stage names the scam type, so each flagged message carries both a verdict and the kind of fraud it resembles.

2. Data

Only one public corpus separates commercial spam from smishing, and after cleaning it leaves 379 real smishing messages. That class is the project's binding constraint. Each source has a fixed role so that none can leak into the wrong place.

Source	Messages (raw)	After dedup.	Role
Mendeley SMS Phishing	5,971 (4,844 ham, 489 spam, 638 smishing)	5,823	Core three-class set; the only real source of smishing
UCI SMS Spam Collection	5,574 (ham, spam)	184	Extra ham and spam; almost all of it is already in Mendeley
NUS SMS Corpus (Singapore senders)	10,116 ham	8,948	Held out to measure false alarms on local personal messages
Singapore scam set (collected)	60 scams, 5 legitimate	65	Evaluation only (section 5)

A fourth corpus, published as LLM-generated scam messages, turned out to be mostly reworded copies of the other corpora; over repeated splits it made no reliable difference, so it is excluded.

Deduplication across the corpora removed 13,595 of 31,852 rows: they are built on each other (Mendeley re-labels UCI, and UCI absorbed part of NUS). 303 identical messages carry different labels across corpora, almost all `uci=spam` against `mendeley=smishing`. That is the spam/smishing conflation this project exists to undo, so the finer Mendeley label wins.

The Singapore scam set was collected from Singapore Police Force advisories (43 scams, all 5 legitimate messages), Mothership (8) and archived ScamAlert stories (9), dated 2018 to 2026. Most sources publish these as screenshots, so 51 of the 60 scam texts were transcribed from the screenshots with AI assistance, and each is marked so in the file.

3. Preprocessing pipeline

The pipeline makes sure the model is never tested on a message, or a near-copy of a message, that it trained on. It runs in five steps (`src/preprocess.py`).

1. **Load and unify.** One loader per corpus emits the same three columns: `text`, `label`, `source`. All files are read as UTF-8; latin-1 would turn every "£" into "Â£", a cue no real incoming message carries.
2. **Deduplicate before splitting.** Messages are matched on a key that ignores case, spacing and punctuation, and conflicting labels resolve to the finer one.
3. **Normalise, keeping two representations.** `text_clean` replaces URLs, emails, phone numbers (including 8-digit Singapore numbers), amounts and numbers with placeholder tokens for word features, because their presence is predictive even when the value is not. The raw text is kept for character features, since digits, case and punctuation are what obfuscation abuses (`DB5`, `acc0unt`).
4. **Split by template, then remove near-copies.** Scam campaigns send one template with a different number or amount each time, so messages are grouped by normalised text and each group goes to one split, stratified by label (80/20, then 85/15 for validation); a row-level split would put 27% of test smishing beside a sibling in training. Templates still miss reworded variants and the same campaign sent with a different phone number. So any training message whose character n-gram similarity to a validation or test message is 0.7 or more is removed, on either the raw or the number-blanked text (cosine over 3–5 character TF-IDF, with weights fitted once on the whole corpus). At that level every pair inspected was the same message with a word or number changed. An independent check with its own weights finds no pair above 0.75.
5. **Keep local data clear.** The same check removes any Singapore shift or scam message that resembles a training message, so both local sets measure unseen text.

Split	Messages	Ham	Spam	Smishing
Train	3,571	3,127	245	199
Validation	735	595	61	79
Test	1,191	992	98	101
Singapore shift (NUS)	8,616	8,616	0	0

Validation and test are not deduplicated against each other: 21 of the 101 test smishing messages nearly copy a validation message. The model never trains on validation, so this is not a training leak, but the two sets are less independent than fresh samples would be. With 101 smishing messages in test, one misclassification moves recall by a full point, so smishing recall is always reported with a bootstrap confidence interval.

4. Models and selection

The served model is a calibrated linear SVM on character n-grams. Its validation macro F1 is within noise of the best model's, and it is the only model that flags under 1% of ordinary Singaporean messages. Four models were fitted, cheapest first, and scored on validation only; the test set was used once, by the selected model.

Model	Features	Val. macro F1	Smishing recall	Smishing precision	False alarms on SG ham
Multinomial Naive Bayes	word counts	0.890	0.810	0.901	1.2%
Logistic regression	word TF-IDF (1–2 grams)	0.913	0.899	0.910	2.8%
Logistic regression	word + character TF-IDF	0.907	0.886	0.897	1.3%
Calibrated linear SVM	**character TF-IDF (2–5 grams)**	**0.894**	**0.873**	**0.896**	**0.24%**

- **Selection rule.** Highest validation macro F1, with smishing recall as the tie-break, among models that flag at most 1% of the Singapore shift set, applied in code. The four models are within 0.03 macro F1 of each other, inside the noise of 79 validation smishing messages, but differ up to tenfold on false alarms, which macro F1 cannot see. The 1% cap was set with these validation results in view.
- **Class imbalance.** Every model except Naive Bayes uses balanced class weights, so errors on the rare smishing class count as much in training as errors on the common classes.
- **Why characters.** Smishing hides in spellings, digits and link shapes that word tokens destroy; character n-grams keep them, and misspellings such as "DB5" and "acc0unt" are still caught (section 5).
- **Calibration.** The SVM is wrapped in sigmoid calibration so it outputs probabilities: the bot shows a confidence, and the operating-point comparison needs one.

Operating point: argmax. The model outputs three probabilities, and a decision rule turns them into one verdict. Argmax takes the most probable class; min-cost takes the class with the lowest expected cost under the project's cost matrix, where a missed scam costs 20 and a false scam alarm costs 2. Both were compared on validation and the Singapore shift set before the test set was touched.

Rule	Macro F1	Smishing recall	Smishing precision	False alarms on SG ham
Argmax (served)	**0.894**	**0.873**	**0.896**	**0.24%**
Min-cost	0.760	0.975	0.681	0.67%

Min-cost would catch eight more of the 79 validation scams, but precision falls to 0.68 and false alarms on real local messages nearly triple, a cost the matrix has no term for. For a checker people must trust, the served rule is argmax (`config.yaml`). A cost matrix is only as good as the costs it leaves out.

5. Results

On the held-out test set the model catches 73.3% of smishing at 80.4% precision, and no smishing message passed as legitimate. On real scams sent to people in Singapore it catches 48%, and that gap is the project's most important finding.

Test metric	Value
Smishing recall	**0.733 (95% CI 0.648–0.819)**
Smishing precision	0.804
Macro F1	0.825
Accuracy	0.953
Brier score (0 is perfect)	0.066
False alarms on Singaporean ham (NUS)	0.24% (21 of 8,616)

True ↓ / predicted →	Ham	Spam	Smishing
Ham (992)	991	1	0
Spam (98)	10	70	18
Smishing (101)	0	27	74

Almost all error sits on the spam/smishing boundary. A scam called spam still reaches the user with a warning, so the costly miss, a scam marked safe, did not occur in test.

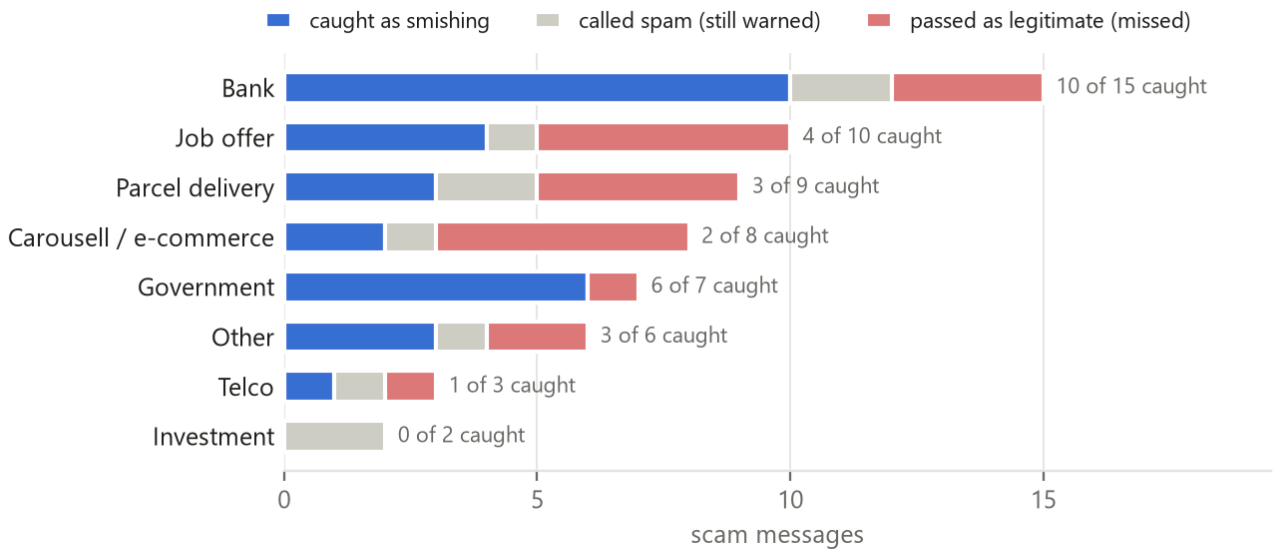
The test figure is the representative one. Validation recall (0.873) is higher than test (0.733) because one split of about 80 smishing messages is noisy. Over six repeated template splits of the non-test data, the same model's recall ranges from 0.54 to 0.91 and averages 0.71, in line with test (`reports/ablation_repeated.csv`).

Confidence is reliable at the extremes on familiar text. Of test messages given about 93% smishing probability, 91% were smishing, and of those given about 1%, 1% were. The middle bins hold only 17–30 messages each and are noisier (`reports/calibration_test.csv`). This calibration holds for text like the training data, not for real local scams, where the model is often confidently wrong.

Real Singapore scams

The Singapore scam set (section 2) was scored once and never used for training or tuning. The model catches 29 of 60 scams (48%) and warns on 39 (65%).

Link-based scams are more often caught; chat-style scams mostly pass



`reports/sg_scams_predictions.csv` · 60 scam messages, 2018–2026

- **The model learned what a scam SMS with a link looks like.** Scams carrying a link were caught 19 times out of 32; scams without one, 10 out of 28.
- **Conversational scams pass as chat.** Carousell fake buyers, job recruiters and a kidnap threat read like personal messages, and personal messages are what the training ham is made of.
- **Legitimate local transactional messages are flagged.** 4 of the 5 genuine SingPost notices and bank OTPs were called spam or smishing. The training ham is almost entirely personal chat, so an official OTP looks more like a scam than like ham. Five messages is a small sample, but the direction is clear.

Robustness probes

Constructed variants of a message test behaviour the test set cannot (`src/robustness.py`). They are reported as behaviour, not accuracy: 8 of 12 get the verdict a user would want. Two failures are link-related, and two are chat-style scams with no link at all.

Probe	Example	Verdict (p smishing)
Misspelling	Y0ur DB5 acc0unt has been l0cked. Verify now at http://dbs-sg-verify.top	Smishing (0.97) ✓
Link with `http://`	Your parcel is held at customs. Pay the fee at http://sgpost-clear.co/pay	Smishing (0.95) ✓
Same link, bare domain	…Pay the fee at sgpost-clear.co/pay	No scam signals (0.18) ✗
Harmless news link	check out this article https://www.straitstimes.com/singapore/abc	Smishing (0.73) ✗
Chat-style scam	Hi mum this is my new number, my phone broke. Can you PayNow me $500 urgently	No scam signals (0.03) ✗
Job scam	Hi, I am Jessica from Lazada HR. Part time job, earn $300-$800 daily. Interested?	No scam signals (0.02) ✗

6. Explainability

Every spam or scam verdict comes with the words that drove it, found by LIME and shown to the user as red flags. A user who sees why a message was flagged learns to spot the next one unaided, which makes the explanation part of the product, not just a report figure.

LIME deletes random subsets of words from the message, re-scores each variant with the real model, and fits a small weighted linear model to those scores; each word's weight is its estimated push towards the verdict. Deleting many words at once lets it explain a 95%-confident prediction, where removing any single word moves nothing.

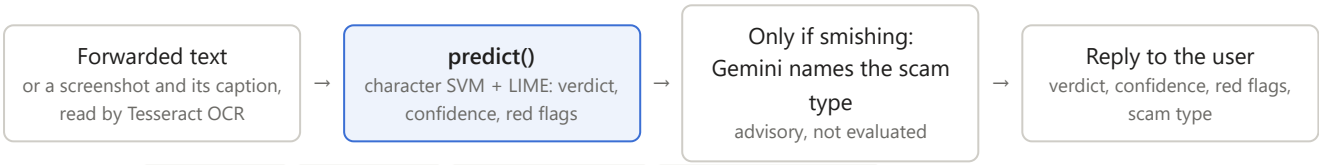
Message	Verdict	Red flags (LIME)
Your DBS account is locked. Verify at http://bit.ly/x9	Likely scam (97%)	http://bit.ly/x9, Verify, account
URGENT! You have won a 1 week FREE membership in our £100,000 Prize Jackpot! Txt the word: CLAIM to No: 81010	Marketing / spam (56%)	Txt, FREE, week, word
WIN a FREE iPhone! Txt WIN to 85233 now. T and Cs apply	Marketing / spam (99%)	FREE, Txt, apply, WIN, Cs
Eh you reaching already? I order first ah	No scam signals found (97%)	none, plus a reminder that this is not a guarantee

Tokens are split on whitespace so a link stays whole and is shown as one flag. Function words such as "your" and "our", and bare numbers such as an OTP, are filtered out. Explanations are seeded, so the same message always yields the same flags, and they take under half a second. LIME explains the model's behaviour, not ground truth: a weak word such as "week" or "Cs" can appear when it co-occurs with the real cue.

What the model relies on overall. The SVM's averaged linear weights give the global picture (`reports/top_features.csv`). The strongest smishing cues are UK premium-rate number prefixes ("08…", "087…", "09…") and "http". The model has learned what a scam SMS in these mostly British and Indian corpora looks like, which explains both the robustness results and the gap on Singapore scams.

7. The application

The product is a forward-it-to-check bot: a user forwards a suspicious message, or a screenshot with or without a caption, and gets a verdict, a confidence and the red flags back within seconds. Telegram is the demo channel because a bot there needs no business verification. A deployment would put the same `predict()` behind the bank's app or WhatsApp Business, where ScamShield's bot already runs.



serving path · `app/bot.py`, `src/ocr.py`, `src/predict.py`, `src/scam_type.py`

Only the highlighted step is evaluated. The trained classifier makes every decision. A zero-shot Gemini call runs only on messages the classifier already called smishing, and adds a likely scam type (parcel, bank impersonation, job offer…) with the phrases that indicate it. Scam-type labels do not exist as public labelled data, so this stage is advisory and no metric depends on it. As a spot check, on the 29 Singapore scams the classifier caught, the Gemini label matched the hand label 24 times (`reports/scam_type_eval.csv`); the misses were near neighbours, such as a Carousell scam labelled as a parcel scam. The gate lives inside `scam_type.enrich()`, not in the caller, and a test checks that a legitimate message never reaches it. Each model is given 15 seconds before a fallback model is tried, and the verdict is sent whether or not Gemini answers.

- **No false all-clear.** A message with no scam signals is never called "safe". The reply says it is not a guarantee and tells the user to check any request for money, a code or a login with the sender on a number they already trust, because about half of real local scams get this verdict.
- **Screenshots.** Users forward screenshots more readily than they retype. Tesseract reads the image, chat furniture (timestamps, "Delivered") is stripped, and the text, together with any caption, goes through the same `predict()`.
- **Privacy.** The bot logs a label, a confidence and a message length, never the message body or the bot token. Legitimate messages never leave the machine. Flagged messages do go to Google, and on the free tier Google may use them to improve its models; a billed account or a locally hosted model would close that gap.
- **Degrades, never fails.** Without a Gemini key the bot still answers with the classifier's verdict; without Tesseract it uses the caption or asks the user to paste text instead.

8. Limitations, risks and scope

The largest risk is that the training smishing is not Singaporean: it is British and Indian in phrasing, brands and phone-number formats, and no public local smishing corpus exists. Section 5 measures what that costs on real local scams.

- **False reassurance.** About half of real local scams get "no scam signals found", often with high confidence. The reply carries a warning, but a user may still read it as an all-clear; this is the main reason the prototype is not ready for customers.
- **Small smishing class.** 379 real smishing messages remain after cleaning: 199 to train on, 79 for validation and 101 for test. The 95% confidence interval on test recall is about nine points either side.
- **Selection on one small validation split.** There is no hyperparameter search, the models are within noise of each other, and single-split recall varies from 0.54 to 0.91. The 1% false-alarm cap was set with the validation results in view, so the Singapore false-alarm rate is a guardrail rather than an untouched estimate. The test set played no part in any choice.
- **The false-alarm figure is for personal chat.** The 0.24% comes from Singaporeans' personal messages collected before 2015, and it is closer to the training data than it looks: UCI's legitimate messages were partly drawn from the same NUS corpus. Transactional messages, the kind a bank's customers actually receive, were flagged in 4 of 5 cases.
- **Adversarial evasion.** Character n-grams catch misspellings like "DB5", but a scam link without `http://` evades the model, and an ordinary message with a news link is flagged.
- **Concept drift.** Scam wording changes faster than any static training set. A deployment would need periodic retraining on newly reported scams and monitoring of the verdict mix.
- **OCR errors land upstream of the classifier.** A misread character becomes a feature the model never trained on; dark mode and low resolution make this worse.
- **Debatable labels on the boundary.** Some source labels between spam and smishing are themselves arguable (premium-rate prize texts are smishing in Mendeley and spam in UCI).
- **Privacy.** Flagged messages are sent to Google for the scam-type label (section 7). A scam checker that ships users' messages to a third party has a tension in it, which should be stated to users.

Next version. The fix for the local gap is local training data: conversation-stage scams labelled smishing, and genuine transactional messages from banks and SingPost labelled ham. Public data of this kind is very small: most local scam examples are published only as screenshots, many chat scams circulate in Chinese, and genuine bank notifications are rarely published at all. A bank's own fraud reports are the natural source, which is what the pilot in section 1 pays for. Any such data must be kept separate from the Singapore scam set, which has to stay unseen to remain a fair test.

Out of scope, deliberately: email phishing (headers, SPF/DKIM and attachments are a different feature space), phishing website detection, voice and in-person scams, live WhatsApp interception (no read API), and automatic on-device SMS filtering.

9. Reproducing the results

Every number in this writeup is regenerated by one command, `python main.py --skip-download`, from the corpora shipped in `data/raw/`; `python -m src.train` alone regenerates the model results from the shipped `data/processed/` splits. All settings (labels, paths, split ratios, seed, near-copy threshold, cost matrix, selection cap, operating point) live in `config.yaml`; no module hardcodes a data path. `requirements-lock.txt` pins the exact library versions the saved model was trained with.

Command	What it does
`python -m src.download`	Fetches UCI, NUS and the OCR language data; lists the Mendeley files that must be downloaded by hand
`python -m src.preprocess`	Deduplicates, normalises, writes the template-aware splits and removes near-copies
`python -m src.train`	Fits the four models, selects on validation, evaluates test once, saves `artifacts/model.joblib`
`python -m src.ablation`	Compares training with and without the excluded corpus, on validation and over six repeated splits
`python -m src.robustness`	Runs the constructed probes: misspellings, link form, harmless links, chat-style scams
`python -m src.predict "message"`	One verdict with confidence and red flags
`python -m src.ocr screenshot.png`	Reads a screenshot, then classifies it
`python -m src.scam_type --evaluate`	Compares the Gemini scam type with the hand labels on the caught Singapore scams (needs `GEMINI_API_KEY`)
`python -m app.bot`	Runs the Telegram bot (keys in `.env`; see `.env.example`)
`python -m pytest tests`	17 checks, including that no template or near-copy is shared between training and validation/test (on the real splits), that the LLM never sees an unflagged message, and that a screenshot's caption is classified

`notebooks/walkthrough.ipynb` walks through the same pipeline with tables and figures; `python build_notebook.py` regenerates and executes it. The code is reusable through one entry point, `predict()`, which returns a label, a confidence, per-class scores and red flags; the bot, the OCR path and the notebook all call it.

References

1. Singapore Police Force (2026). *Annual Scam and Cybercrime Brief 2025.* https://www.police.gov.sg/-/media/SPF/Advisories/Scams/Annual-Scam-and-Cybercrime-Brief-2025.pdf
2. Singapore Police Force (2026). *Mid-Year Scam and Cybercrime Brief 2026.* https://www.police.gov.sg/-/media/SPF/Media-Room/Statistics/Mid-Year-Scam-and-Cybercrime-Brief-2026/Mid-Year-Scam-and-Cybercrime-Brief-2026.pdf
3. Monetary Authority of Singapore and Infocomm Media Development Authority (2024). *Guidelines on Shared Responsibility Framework*, effective 16 December 2024.
4. ScamShield (n.d.). *About the ScamShield Bot.* https://www.scamshield.org.sg/about-bot